Trabajo Práctico Integrador de Lógica y Teoría de Grafos

Planificación de mesas de examen

Integrantes: Franco Reyes, Geronimo Chavarri, Ignacio Tasada

Fecha de entrega: 08/11/2026

Repositorio: https://github.com/FrancoReyes3223/Pic-chavarri-reyes-tasada

1. Problema

"La secretaría académica de una facultad tiene que armar el cronograma de mesas de examen. Si dos materias que comparten muchos estudiantes se rinden en el mismo turno, esos estudiantes tienen que elegir cuál rendir. La secretaría quiere un cronograma que evite esos choques y que use la menor cantidad de turnos posible, porque cada turno extra alarga el período de exámenes."

Para tomar esa decisión haría falta saber, para cada par de materias, cuántos estudiantes van a querer rendir las dos en el mismo período. Ese número no se conoce de antemano: depende de cuántos alumnos cursan cada materia, de si son del mismo año y carrera, y de cuántos compartieron en períodos anteriores. Por eso lo estimamos con un modelo entrenado sobre datos históricos.

El problema se modela naturalmente como un grafo de conflictos no dirigido:

- **Nodos:** las 45 materias que se rinden en el período.
- **Aristas:** un par de materias que no pueden compartir turno, porque el modelo predice que el conflicto supera un umbral.
- **Peso de la arista:** la cantidad de estudiantes afectados que predice el modelo.

Con este modelo, asignar turnos es colorear el grafo: cada color es un turno y dos nodos unidos por una arista no pueden tener el mismo color. La menor cantidad de turnos posible es el número cromático del grafo. Como pide el enunciado, usamos un algoritmo greedy, que da una solución válida rápido aunque no siempre la óptima.

El recorrido completo del sistema es:

pares de materias → modelo de regresión (estudiantes afectados) → grafo de conflictos (arista si la predicción ≥ umbral) → coloreo greedy (color = turno) → API → frontend

2. Decisiones de diseño

- **Regresión y no clasificación.** El enunciado permite predecir si hay conflicto (clasificación) o cuántos estudiantes se ven afectados (regresión). Elegimos regresión porque el número sirve para dos cosas a la vez: decidir si hay arista y darle un peso. Además, el umbral se puede mover sin reentrenar el modelo. Con clasificación, el umbral quedaría fijo dentro del dataset.

- **El umbral se aplica al armar el grafo.** No está en el dataset ni en el modelo: es un parámetro del backend que el usuario elige con un slider en el frontend. Representa cuántos estudiantes afectados está dispuesta a tolerar la secretaría.
- **Dataset sintético generado con un script** (`backend/data/generar_dataset.py`), con reglas declaradas. Lo explicamos en la sección 3.
- **Partición temporal** para evaluar: entrenamos con los períodos más viejos y testeamos con los más nuevos, que es como se usaría el sistema en la realidad.
- **El test no se usa para elegir nada.** Los hiperparámetros y el modelo ganador se eligen con validación cruzada sobre train. El test se mira una sola vez, al final.
- **El coloreo greedy lo implementamos nosotros.** `networkx.greedy_color` solo se usa para comparar y validar el resultado.
- **Scripts .py y no notebooks**, para que el entrenamiento se pueda correr de punta a punta con un comando.
- **Código simple:** evitamos construcciones que no vimos en clase (por ejemplo, comprensiones de listas), para poder explicar cada línea en la defensa.

3. Dataset

3.1 Por qué sintético

No encontramos un dataset público con inscripciones por materia y estudiantes compartidos entre pares de materias, y los datos reales de una facultad son privados. Por eso generamos un dataset sintético con un script en Python. El script es reproducible (usa una semilla fija, 42), se puede volver a correr con otra semilla y deja escritas las reglas que usa.

3.2 Reglas de generación

Estas son las reglas que usa el generador, tal como están en el docstring del script:

- **R1. Plan de estudios:** 3 carreras (Sistemas, Industrial y Civil) × 5 años × 3 materias por año = 45 materias, que forman 990 pares.
- **R2. Inscriptos por materia y período:** inscriptos = base_por_año × tamaño_materia × variación_período.
    - La base cae con los años: 160, 120, 90, 70 y 55 alumnos para los años 1 a 5.
    - El tamaño de cada materia es fijo y sale de una Uniforme(0.7, 1.3).
    - La variación de cada período sale de una Normal(1, 0.10).
- **R3. Afinidad del par** (qué fracción de alumnos comparten), según la relación entre las materias:
    - Misma carrera y mismo año: 0.40. Con 1 año de diferencia: 0.15. Con 2 años: 0.05. Con 3 o más: 0.02.
    - Distinta carrera y mismo año, en 1.º o 2.º año (ciclo básico común): 0.12. En 3.º año o más: 0.03.
    - Distinta carrera y distinto año: 0.01.
    - Esa afinidad se multiplica por un efecto propio de cada par, que sale de una LogNormal(0, 0.25) y no cambia con el tiempo. Representa razones que no observamos por las que algunos pares comparten más o menos alumnos.
- **R4. Estudiantes compartidos en un período:** Poisson(afinidad × mínimo de los inscriptos de las dos materias).

- **R5. Variable a predecir:** cant_estudiantes_afectados = Binomial(compartidos, p), con p = 0.55 si las materias son del mismo año y 0.35 si no. Es la cantidad de los alumnos compartidos que quieren rendir las dos materias en el mismo período.
- **R6. Feature histórica:** cant_estudiantes_compartidos_historico es el promedio de compartidos en los 3 períodos anteriores. Los compartidos del propio período no se exportan, así el modelo no ve información del futuro.

Según estas reglas, lo que el modelo debería descubrir es aproximadamente **afectados ≈ p(mismo_anio) × compartidos_historico**. Es esperable que lo encuentre, porque nosotros pusimos esa relación en los datos. El resto es ruido (Poisson, Binomial y la variación de inscriptos) que ningún modelo puede predecir, y por eso ningún modelo puede tener error cero.

3.3 Archivos y features

El script genera tres archivos CSV:

- `materias.csv`: las 45 materias, con su id, nombre, carrera y año. Son los nodos del grafo.
- `pares_historico.csv`: 990 pares × 8 períodos = 7920 filas, con las features y el target. Es el que se usa para entrenar y evaluar.
- `pares_proximo_periodo.csv`: las features de los 990 pares para el período que hay que planificar, sin el target. Sobre este archivo predice el backend para armar el grafo.

Feature	**Qué es**	**Por qué sirve**
cant_estudiantes_compartidos_historico	Promedio de alumnos compartidos en los 3 períodos anteriores	Es la mejor señal: si compartieron muchos antes, van a compartir muchos ahora
mismo_anio	1 si las dos materias son del mismo año	Cambia la probabilidad de querer rendir las dos (0.55 contra 0.35)
misma_carrera	1 si son de la misma carrera	Los alumnos comparten mucho más dentro de una carrera
diferencia_anios	Años de diferencia entre las materias	Cuanto más lejos, menos alumnos en común
anio_min	El menor de los dos años	Distingue el ciclo básico común (años 1 y 2)
inscriptos_prom_1 y _2	Promedio de inscriptos de cada materia en los 3 períodos anteriores	Una materia más grande puede tener más alumnos en común

`periodo`, `materia_id_1` y `materia_id_2` no son features. Los ids no tienen un significado numérico: si los usáramos, el modelo podría memorizar pares en vez de aprender por qué chocan. El período solo sirve para partir los datos.

El target tiene una distribución muy asimétrica: la media es 2.23, la mediana es 0 y el máximo es 67 (jijiji). El 53 % de los pares no tiene ningún estudiante afectado. Los valores altos aparecen casi solo en pares de la misma carrera y el mismo año. La feature más correlacionada con el target es la histórica (0.929), seguida de mismo_anio (0.518).

4. Entrenamiento del modelo

Todo el entrenamiento está en `backend/entrenamiento.py`.

4.1 Partición de los datos

Usamos una partición temporal: train son los períodos 1 a 6 (5940 filas) y test los períodos 7 y 8 (1980 filas). En la realidad el modelo se entrena con el pasado y se usa para predecir el próximo período, así que el test tiene que ser posterior al train. Con una partición al azar, el mismo par de materias aparecería en train y en test en períodos vecinos, y el error de test saldría más bajo de lo que va a ser en la práctica.

4.2 Modelos y selección de hiperparámetros

Comparamos tres de los modelos permitidos por el enunciado:

- **Regresión lineal**, sin hiperparámetros.
- **Árbol de decisión**, probando `max_depth` de 1 a 20.
- **KNN**, probando k = 3, 5, 10, 15, 20, 30 y 50.

Los hiperparámetros y el modelo ganador se eligen sobre train, con `GroupKFold` por período: hay 6 folds y cada uno deja afuera un período entero para validar. Así la validación también respeta que se predice un período que el modelo no vio. El criterio es el MAE promedio de validación.

KNN va dentro de un pipeline con `StandardScaler`, porque calcula distancias entre filas. La regresión lineal y el árbol no dependen de la escala de las features, así que van sin escalar.

4.3 Métricas

- **MAE** (error absoluto medio): el error promedio en cantidad de alumnos.
- **RMSE**: castiga más los errores grandes, que en este problema son los que importan (los pares con muchos alumnos en común).
- **R^2**: qué parte de la variación del target explica el modelo.

4.4 Resultados

Modelo	Hiperparámetro	MAE CV	MAE train	MAE test	RMSE test	R^2 test
Regresión lineal	—	1.042	1.040	1.069	1.929	0.876
Árbol de decisión	max_depth = 5	0.989	0.916	1.038	1.984	0.869

KNN (ganador)	k = 15	0.946	0.887	1.021	1.855	0.886

KNN con k = 15 tiene el menor MAE de validación cruzada, y por eso es el modelo que usa el backend. En test también es el mejor en las tres métricas, pero eso lo vimos después de elegirlo. Las diferencias entre los tres modelos son chicas: el target depende casi solo de la feature histórica multiplicada por un factor que cambia según mismo_anio. La regresión lineal no puede representar exactamente esa multiplicación (suma los efectos), y por eso queda un poco por detrás.

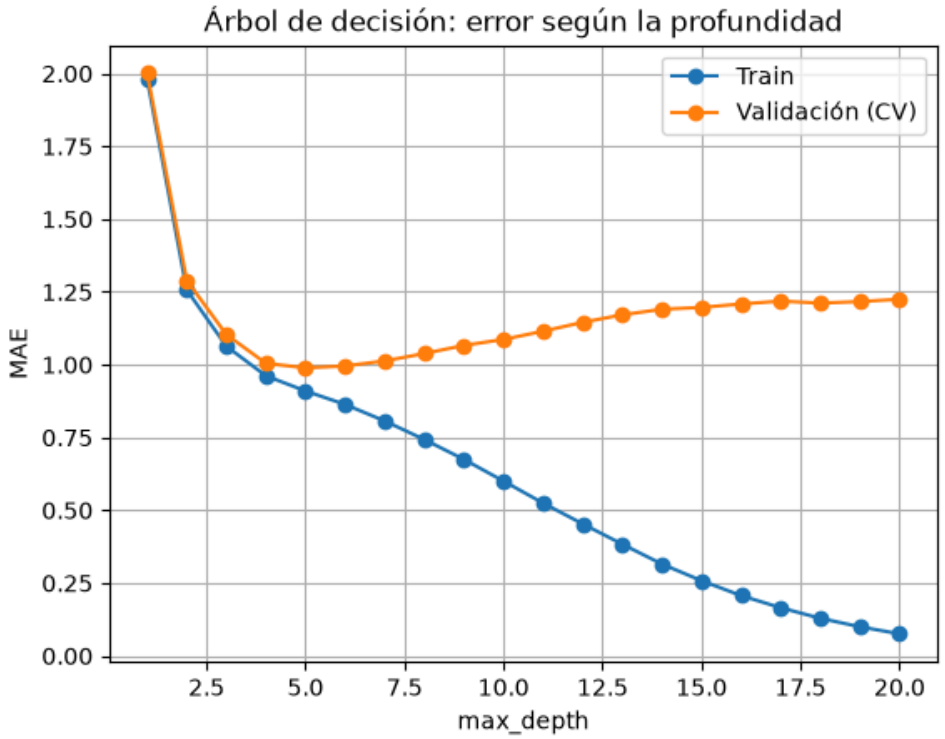
4.5 Overfitting y underfitting

Para ver el overfitting usamos la curva del árbol según su profundidad, con los errores de train y validación de la validación cruzada:

max_depth	**1**	**2**	**3**	**5**	**8**	**12**	**20**
MAE train	1.982	1.257	1.064	0.910	0.743	0.453	0.076
MAE validación	2.001	1.290	1.104	0.989	1.038	1.146	1.225

- Con profundidad 1 o 2 los dos errores son altos y parecidos: el árbol es demasiado simple y no llega a aprender la relación (**underfitting**).
- Con profundidad 5 el error de validación es el mínimo, y por eso se elige ese valor.
- Desde ahí, el error de train sigue bajando y el de validación sube. Con profundidad 20 el MAE de train es 0.076 y el de validación 1.225: el árbol memoriza el ruido de los datos de entrenamiento (**overfitting**).

El KNN elegido no muestra overfitting: su MAE de train (0.887) y el de test (1.021) son parecidos.



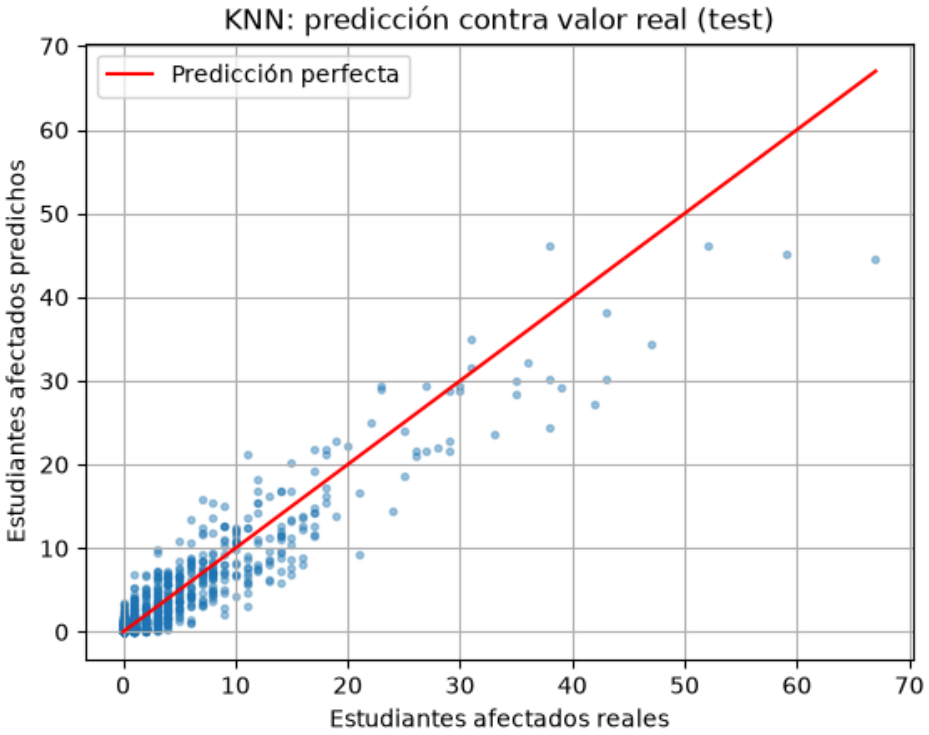
4.6 Análisis de errores

Analizamos los errores de KNN en el test. El error promedio con signo (predicción menos real) es −0.10: el modelo tiende a predecir un poco de menos.

Mismo año	**Misma carrera**	**Pares**	**Afectados promedio**	**MAE**
No	No	1080	0.3	0.43
No	Sí	540	2.3	1.28
Sí	No	270	4.1	1.69
Sí	Sí	90	20.6	4.63

El error crece con la cantidad de estudiantes en común. En los pares de distinta carrera y distinto año casi no hay alumnos afectados y el modelo acierta (MAE 0.43). En los pares de la misma carrera y el mismo año, que son los que más importan para el cronograma, el MAE llega a 4.63.

El gráfico de predicción contra valor real muestra otra limitación: KNN no predice más de unos 46 afectados (el máximo real en test es 67(jijiji)). Como la predicción es el promedio de los 15 vecinos más cercanos, nunca puede superar los valores más altos que vio en train, y en los extremos queda por debajo.



Por último, medimos qué efecto tienen estos errores en el grafo. Para cada período de test comparamos el grafo que arma el modelo con el que se armaría con los valores reales:

Período	Umbral	Aristas reales	Aristas predichas	Sobran	Faltan
7	2	275	208	17	84
7	5	138	128	18	28
7	10	67	53	9	23
8	2	276	208	23	91
8	5	133	129	19	23
8	10	63	55	8	16

En todos los casos faltan más aristas de las que sobran, y la diferencia es mayor con umbrales bajos (con umbral 2 en el período 8, faltan 91 y sobran 23). Esto es coherente con que el modelo predice un poco de menos. Las dos clases de error no cuestan lo mismo: una arista que sobra agrega una restricción innecesaria y puede costar un turno de más, pero una arista que falta puede dejar en el mismo turno dos materias que en realidad chocan.

5. Algoritmo de grafos

5.1 Construcción del grafo

El grafo se arma en `backend/app/grafo.py` con networkx:

1. Se cargan el modelo y `pares_proximo_periodo.csv`, y se predicen los estudiantes afectados de los 990 pares.
2. Se agregan las 45 materias como nodos, aunque alguna quede sin conflictos. Una materia sin aristas también necesita un turno.
3. Se agrega una arista por cada par cuya predicción sea mayor o igual al umbral, con la predicción como atributo `weight`.

El umbral tiene que ser mayor que 0. Con umbral 0 todos los pares tendrían arista, porque toda predicción es mayor o igual a 0, y el grafo dejaría de tener sentido.

5.2 Coloreo greedy

El coloreo está en `backend/app/coloreo.py`, en la función `colorear_grafo`:

1. Se ordenan las materias de mayor a menor grado ponderado, es decir, la suma de los pesos de sus aristas (el total de estudiantes en conflicto). Los empates se resuelven por id, para que el mismo umbral dé siempre el mismo cronograma.
2. Se recorren las materias en ese orden. Para cada una se arma el conjunto de turnos que ya usan sus vecinas (las materias con las que tiene conflicto).
3. De los turnos que ya existen y no usa ninguna vecina, la materia va al que tiene menos materias. Si hay empate, va al de número más bajo.
4. Si todos los turnos existentes tienen alguna vecina, se abre un turno nuevo.

Las materias con más conflictos van primero porque son las más difíciles de ubicar: si quedaran para el final, sería más probable que todos los turnos estuvieran ocupados por alguna vecina y hubiera que abrir uno nuevo.

Elegir el turno libre con menos materias (en vez del turno libre con número más bajo, que es lo más común) hace que los turnos queden parejos. Con umbral 5, por ejemplo, salen 9 turnos de 5 materias cada uno. Con "el primer turno libre", los primeros turnos se llenan y los últimos quedan con una o dos materias, y un cronograma así es peor para la secretaría.

5.3 Verificación y resultados

Verificamos el coloreo de dos formas. La función `verificar_coloreo` comprueba que todas las materias tengan turno y que ninguna arista una dos materias del mismo turno. Además, comparamos la cantidad de turnos con la que obtiene `networkx.greedy_color` (estrategia largest_first) sobre el mismo grafo y con el tamaño del mayor subgrafo completo, que es la cota inferior. Esta tabla la imprime `python coloreo.py`:

Umbral	Aristas	Turnos (nuestro greedy)	Materias por turno	Mayor subgrafo completo	Turnos (networkx)	Válido
0.5	439	12	3 a 5	10	11	Sí
1	328	10	4 a 5	9	9	Sí
2	208	10	4 a 5	9	12	Sí
3	180	10	4 a 5	9	11	Sí
5	126	9	5	9	9	Sí
8	72	5	9	5	5	Sí
10	56	4	11 a 12	4	4	Sí
15	28	3	15	3	3	Sí

- Todos los coloreos son válidos.
- Con umbral 5, 8, 10 y 15 la cantidad de turnos es igual al tamaño del mayor subgrafo completo. Como ese tamaño es una cota inferior, en esos casos sabemos que el resultado es óptimo. Probamos además todos los umbrales enteros del slider: de 4 a 30 el resultado siempre es igual a ese tamaño, o sea, óptimo.
- Con umbral 1, 2 y 3 usamos 10 turnos y la cota es 9: como mucho estamos un turno por encima del óptimo.
- Frente a networkx, nuestro greedy usa menos turnos con umbral 2 y 3 (10 contra 12 y 11), la misma cantidad de 4 a 30 y uno más con umbral 0.5 y 1 (12 contra 11 y 10 contra 9). Con umbral 1, networkx llega a 9, que es el óptimo, y nosotros no: es el costo de elegir el turno con menos materias para que los turnos queden parejos.

En cuanto al costo: ordenar las materias lleva $O(n \log n)$, y para cada materia se recorren sus vecinas y los turnos existentes, así que el total es $O(n \log n + m + n \cdot T)$, donde n es la cantidad de materias, m la de aristas y T la de turnos. Con 45 materias el coloreo es instantáneo.

6. Integración entre el modelo y el grafo

La salida del modelo entra al algoritmo de grafos de dos formas:

- **Decide qué aristas existen:** hay arista si la predicción de estudiantes afectados es mayor o igual al umbral.
- **Es el peso de la arista**, y el greedy usa esos pesos para decidir el orden en que ubica las materias.

Esta transformación es razonable porque la predicción mide justamente lo que cuesta poner dos materias en el mismo turno: la cantidad de alumnos que tendrían que elegir cuál rendir. El umbral traduce eso en una decisión de sí o no, y la secretaría elige dónde cortar. Con un umbral bajo se protege a casi todos los alumnos, pero se necesitan más turnos; con uno alto alcanzan menos turnos, a costa de aceptar algunos conflictos chicos.

Si el modelo cambia su predicción, cambian las aristas, el orden del greedy y el cronograma. Lo mismo pasa al mover el umbral en el frontend. Los errores del modelo también pasan al cronograma.

6.1 Arquitectura

- **Backend** (Python, FastAPI) en `backend/app/main.py`. Las predicciones del próximo período se calculan una sola vez, la primera vez que se piden, y se guardan en memoria. En cada pedido solo se arma el grafo con el umbral pedido y se colorea.
- **Frontend** (React con TypeScript y Vite) en `frontend/`. Llama a la API y dibuja el grafo y el cronograma.

Endpoint	**Qué devuelve**
GET /	Estado del servicio (health check)
GET /metricas	La tabla comparativa de modelos y cuál se eligió (el frontend no la usa; se puede ver en /docs)
GET /cronograma?umbral=5	El grafo de conflictos (nodos con su turno y aristas con su peso) y las materias de cada turno

7. Frontend

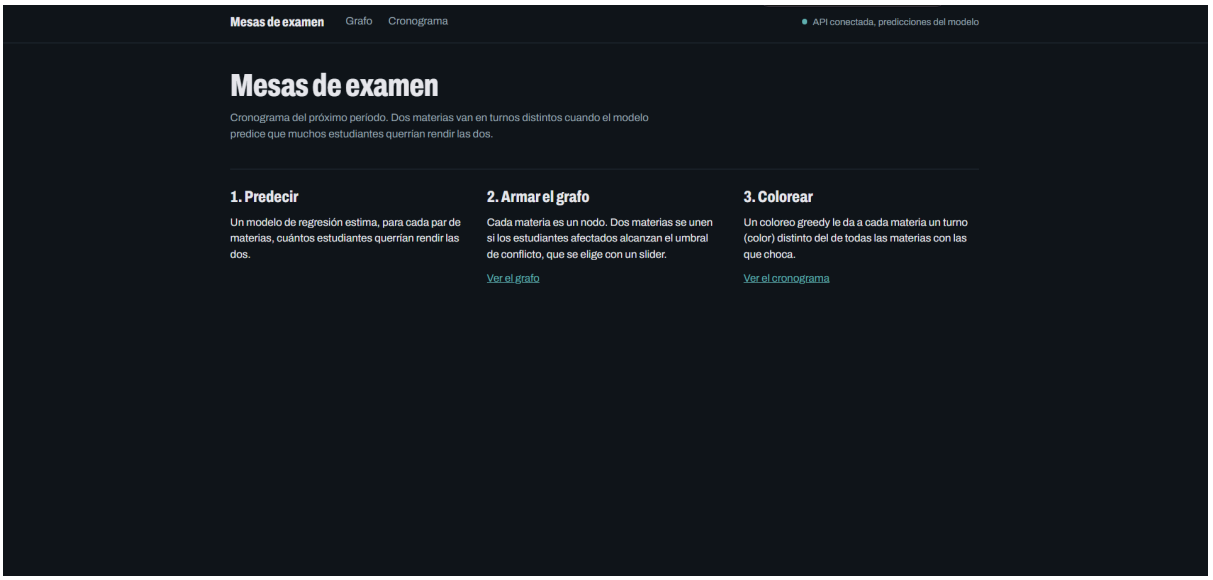
El frontend tiene una página por sección:

- **Inicio (/):** los tres pasos del sistema (predecir, armar el grafo y colorear).
- **Grafo (/grafo):** el slider de umbral, un resumen (materias, conflictos y turnos) y el grafo coloreado por turno, con una gama de colores. El grafo se puede ver de tres formas:
    - **Por cercanía:** una simulación de fuerzas acomoda las materias, así las que tienen muchos conflictos entre sí quedan juntas. Se pueden arrastrar nodos y hacer zoom. No tiene un orden en específico.
    - **Por carrera y año:** una fila por carrera y una columna por año, con posiciones fijas. Muestra por qué hay conflictos: con umbral 5, las 126 aristas unen materias de la misma carrera o del mismo año.
    - **Por turno:** una columna por turno. Muestra el resultado del coloreo: ninguna arista une dos materias de la misma columna.
- **Cronograma (/cronograma):** el mismo slider y una grilla con las materias de cada turno.

El grosor de cada arista es proporcional a la cantidad de estudiantes afectados. El slider va de 1 a 30 en números enteros, y el de /grafo y el de /cronograma comparten el mismo valor.

Del turno 9 en adelante los colores se repiten con rayas, para que no haya dos turnos iguales.

7.1 Capturas de pantalla





Mesas de examen Grafo Cronograma

API conectada, predicciones del modelo

Grafo de conflictos

Cada punto es una materia, pintada con el color de su turno. Una línea une dos materias en conflicto, y es más gruesa cuantos más estudiantes afectados predice el modelo. Dos materias unidas nunca tienen el mismo color.

Umbral de conflicto

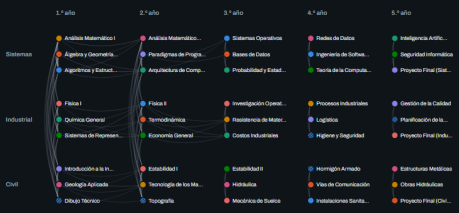
5

Dos materias van en turnos distintos si el modelo predice que al menos 5 estudiantes quieren rendir las dos. Más bajo: más pares en conflicto y más turnos.

Con un umbral de **5** estudiantes, las **45** materias necesitan **9 turnos** para separar los **126** pares en conflicto.

Por cercanía **Por carrera y año** Por turno

Muestra por qué hay conflictos: cada fila es una carrera y cada columna un año, así se ve que chocan las materias del mismo año y de años cercanos. Al cambiar el umbral las materias no se mueven, solo aparecen o desaparecen conflictos.



Qué representa cada color

Color	Turno	Materias
	Turno 1	6
	Turno 2	6
	Turno 3	6
	Turno 4	6
	Turno 5	6
	Turno 6	6
	Turno 7	6
	Turno 8	6
	Turno 9	6

Pasá el mouse por una materia para resaltar sus conflictos.

Mesas de examen Grafo Cronograma

API conectada, predicciones del modelo

Grafo de conflictos

Cada punto es una materia, pintada con el color de su turno. Una línea une dos materias en conflicto, y es más gruesa cuantos más estudiantes afectados predice el modelo. Dos materias unidas nunca tienen el mismo color.

Umbral de conflicto

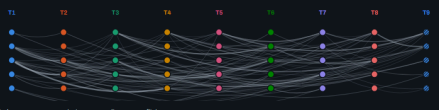
5

Dos materias van en turnos distintos si el modelo predice que al menos 5 estudiantes quieren rendir las dos. Más bajo: más pares en conflicto y más turnos.

Con un umbral de **5** estudiantes, las **45** materias necesitan **9 turnos** para separar los **126** pares en conflicto.

Por cercanía Por carrera y año **Por turno**

Muestra el resultado del coloreo: cada columna es un turno, y ninguna línea une dos materias de la misma columna. Por eso se pueden rendir el mismo día.



Qué representa cada color

Color	Turno	Materias
	Turno 1	6
	Turno 2	6
	Turno 3	6
	Turno 4	6
	Turno 5	6
	Turno 6	6
	Turno 7	6
	Turno 8	6
	Turno 9	6

Pasá el mouse por una materia para resaltar sus conflictos.

Mesas de examen Grafo Cronograma

API conectada, predicciones del modelo

Cronograma

Las materias de un mismo turno se pueden rendir el mismo día: ningún par entre ellas alcanza el umbral.

Umbral de conflicto

5

Dos materias van en turnos distintos si el modelo predice que al menos 5 estudiantes quieren rendir las dos. Más bajo: más pares en conflicto y más turnos.

Con un umbral de **5** estudiantes, las **45** materias necesitan **9 turnos** para separar los **126** pares en conflicto.

Turno 1
6 materias

Algoritmos y Estructuras de Datos
Sistemas, 1.º año

Sistemas Operativos
Sistemas, 3.º año

Ingeniería de Software
Sistemas, 4.º año

Física II
Industrial, 2.º año

Instalaciones Sanitarias
Civil, 4.º año

Turno 2
6 materias

Álgebra y Geometría Analítica
Sistemas, 1.º año

Bases de Datos
Sistemas, 3.º año

Termodinámica
Industrial, 2.º año

Vías de Comunicación
Civil, 4.º año

Proyecto Final (Civil)
Civil, 5.º año

Turno 3
6 materias

Arquitectura de Computadoras
Sistemas, 2.º año

Probabilidad y Estadística
Sistemas, 3.º año

Inteligencia Artificial
Sistemas, 5.º año

Química General
Industrial, 1.º año

Costos Industriales
Industrial, 3.º año

Turno 4
6 materias

Análisis Matemático I
Sistemas, 1.º año

Resistencia de Materiales
Industrial, 3.º año

Procesos Industriales
Industrial, 4.º año

Tecnología de los Materiales
Civil, 2.º año

Obras Hidráulicas
Civil, 5.º año

Turno 5
6 materias

Análisis Matemático II
Sistemas, 2.º año

Redes de Datos
Sistemas, 4.º año

Geología Aplicada
Civil, 1.º año

Hidráulica
Civil, 3.º año

Estructuras Metálicas
Civil, 5.º año

Turno 6
6 materias

Teoría de la Computación
Sistemas, 4.º año

Seguridad Informática
Sistemas, 5.º año

Sistemas de Representación
Industrial, 1.º año

Economía General
Industrial, 2.º año

Estabilidad II
Civil, 3.º año

Turno 7
6 materias

Paradigmas de Programación
Sistemas, 2.º año

Proyecto Final (Sistemas)
Sistemas, 5.º año

Logística
Industrial, 4.º año

Gestión de la Calidad
Industrial, 5.º año

Introducción a la Ingeniería Civil
Civil, 1.º año

Turno 8
6 materias

Física I
Industrial, 1.º año

Investigación Operativa
Industrial, 3.º año

Proyecto Final (Industrial)
Industrial, 5.º año

Estabilidad I
Civil, 2.º año

Mecánica de Suelos
Civil, 3.º año

Turno 9
6 materias

Higiene y Seguridad
Industrial, 4.º año

Planificación de la Producción
Industrial, 5.º año

Dibujo Técnico
Civil, 1.º año

Topografía
Civil, 2.º año

Hormigón Armado
Civil, 4.º año

8. Uso de inteligencia artificial

8.1 Herramientas

Usamos Claude Code y Antigravity. Le dimos contexto del proyecto con un archivo `.claude/CLAUDE.md` (decisiones tomadas, estructura y convenciones) y con tres "skills" propias: una para registrar los prompts en `prompts_ia.md`, otra para regenerar y revisar el dataset y otra para practicar la defensa. Así los tres integrantes trabajamos con el mismo contexto y la IA no volvía a proponer cosas que ya habíamos descartado.

8.2 Para qué la usamos y para qué no

La mayor parte del código la escribió la IA siguiendo nuestras indicaciones. Nosotros decidimos qué hacer, revisamos lo que generó, lo probamos y le pedimos correcciones cuando algo no estaba bien. En detalle:

- **Plan del trabajo:** a partir del enunciado y del repositorio de la materia, propuso un orden de tareas, usar regresión y generar un dataset sintético con un script.
- **Dataset:** escribió el generador con las reglas declaradas. Nosotros fijamos el tamaño (carreras, años y materias) y sacamos la variable de clasificación que había agregado.
- **Entrenamiento:** escribió `entrenamiento.py`. Cuando los resultados de KNN no nos cerraban, le preguntamos si faltaba escalar las features, y de ahí salieron el `StandardScaler` y la selección con validación cruzada. También agregó el análisis de errores.
- **Grafo y coloreo:** Nos recomendo librerias, enseño funciones que podriamos usar de esas librerias y sirvió de corrector sobre las funciones escritas.
- **API:** escribió `main.py` a partir de un prompt detallado con el formato de respuesta acordado. Al principio tenía datos de prueba (mock) para que el frontend pudiera avanzar sin esperar al grafo; cuando el grafo y el coloreo estuvieron listos, lo conectamos y sacamos el mock.
- **Frontend:** Programó la idea del frontend, con el diseño que se le propuso y se fue corrigiendo en múltiples iteraciones. Nosotros elegimos la librería del grafo de fuerzas (`react-force-graph-2d`, aunque recomendaba un SVG propio), decidimos separar las secciones en páginas, quedarnos con las tres vistas.
- **Documentación y scripts:** el README y `proyecto.py`, el script que instala y levanta todo con un solo comando.
- **Explicaciones:** le pedimos que nos explicara conceptos (cómo construir el grafo con networkx) y que nos tomara preguntas de práctica para la defensa.

No la usamos para tomar las decisiones de diseño de la sección 2 ni para elegir el modelo: el modelo ganador sale de la validación cruzada.

8.3 Prompts que nos resultaron útiles

El registro completo está en `prompts_ia.md`. Algunos ejemplos:

- "No me terminan de cuadrar los resultados de KNN, dan demasiado mal aunque el modelo debería funcionar correctamente para el contexto, ¿puede ser que no estén escaladas las features?" De acá salió el escalado de KNN y, además, que dejáramos de elegir el modelo mirando el test.

- "Antes de programar, mostrame 2 o 3 librerías para dibujarlo con sus ventajas y desventajas, y elegimos juntos. Quiero algo simple." Nos sirvió para comparar antes de elegir, en vez de aceptar la primera opción.
- "Tengo que empezar con la construcción del grafo basado en las predicciones del modelo, recomendame librerías y ayudame a construir el grafo, no modifiques código." Lo usamos para entender los pasos antes de escribir nada.
- El prompt que usamos para la API, con el objetivo, los endpoints, el formato del JSON y las convenciones de estilo, mostró que un pedido detallado da un resultado que se integra sin cambios.

8.4 Casos en que la IA se equivocó o propuso algo que no queríamos

Situación	**Cómo nos dimos cuenta**	**Qué hicimos**
Agregó la variable de clasificación hubo_conflicto aunque íbamos a usar regresión	Revisando el código generado	Le pedimos que la sacara
Metió un entrenamiento de modelos dentro del generador de datos	Al generar el dataset nos mostró resultados de over/underfitting que no correspondían a ese paso	Lo sacamos y dejamos el análisis en entrenamiento.py
La primera versión del entrenamiento elegía el modelo y la profundidad del árbol mirando el MAE de test, y no escalaba las features para KNN	KNN daba peor de lo esperable	Pasamos a GridSearchCV con GroupKFold sobre train y StandardScaler en KNN

8.5 Reflexión

La IA nos ayudó sobre todo con la velocidad: en pocos días tuvimos un sistema completo, con un frontend más trabajado del que habríamos hecho solos. También fue útil para entender conceptos y para revisar el trabajo.

No nos ayudó a decidir qué queríamos: cuando el pedido era vago, la IA completaba los huecos con suposiciones, y tuvimos que revisar y corregir. Aprendimos a pedirle que explicara o comparara opciones antes de escribir código.

9. Limitaciones y posibles extensiones

9.1 Limitaciones

- **El dataset es sintético.** El modelo aprende las reglas que nosotros pusimos, así que los buenos resultados no garantizan que funcione igual con datos reales.
- **KNN no extrapola:** no predice más de unos 46 afectados, y en los pares con más alumnos en común predice de menos.
- **Faltan conflictos reales**, sobre todo con umbrales bajos. Esas materias podrían caer en el mismo turno.

- **El greedy no garantiza el óptimo.** Con umbral de 1 a 3 usa 10 turnos y la cota inferior es 9.
- **Los turnos no tienen fecha, horario ni aula.** El sistema no considera la capacidad de las aulas ni restricciones como que un alumno no tenga dos exámenes en días seguidos.
- **El peso solo se usa para el orden del greedy.** Una vez que hay arista, una de 6 alumnos restringe igual que una de 40, y los conflictos por debajo del umbral se ignoran aunque en un mismo turno sumen muchos alumnos.

9.2 Posibles extensiones

- Comparar con otros algoritmos de coloreo, como DSATUR, o resolver el problema exacto (por ejemplo con programación lineal entera) para medir qué tan lejos queda el greedy del óptimo.
- Usar el peso de las aristas cuando no alcanzan los turnos: permitir algunos conflictos chicos y minimizar la cantidad total de alumnos afectados.
- Agregar restricciones reales: cantidad máxima de turnos, capacidad de aulas, separación entre exámenes del mismo año.
- Probar con datos reales de inscripciones.